

Éléments finis d'ordre élevé pour la simulation interactive d'objets déformables avec des méthodes de résolution locales

Bastien Saillant

LIRIS - Équipe Origami

Guillaume Damiand

Directeur de recherche

CNRS

Florence Zara

Maître de conférences, HDR

Université Lyon 1

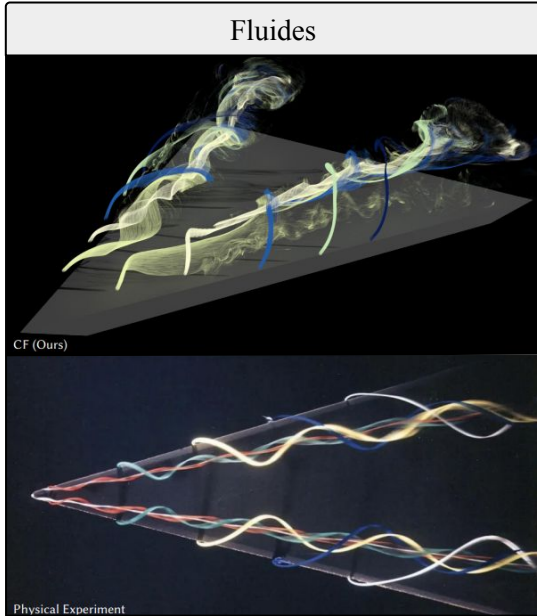
Fabrice Jaillet

Maître de conférences, HDR

Université Lyon 1

Contextes et Motivations

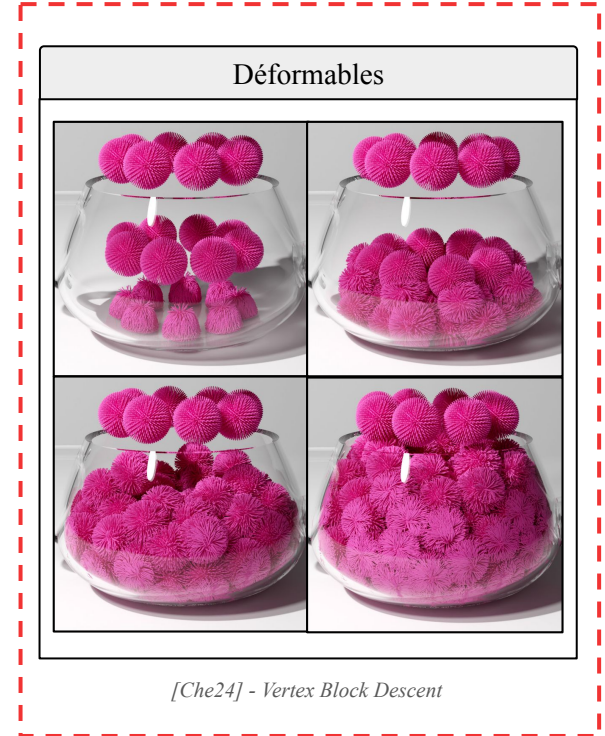
Simulation numérique de phénomènes naturels



[Sin22] - Covector Fluid



[Mül20] - Detailed rigid body simulation with extended position based dynamics

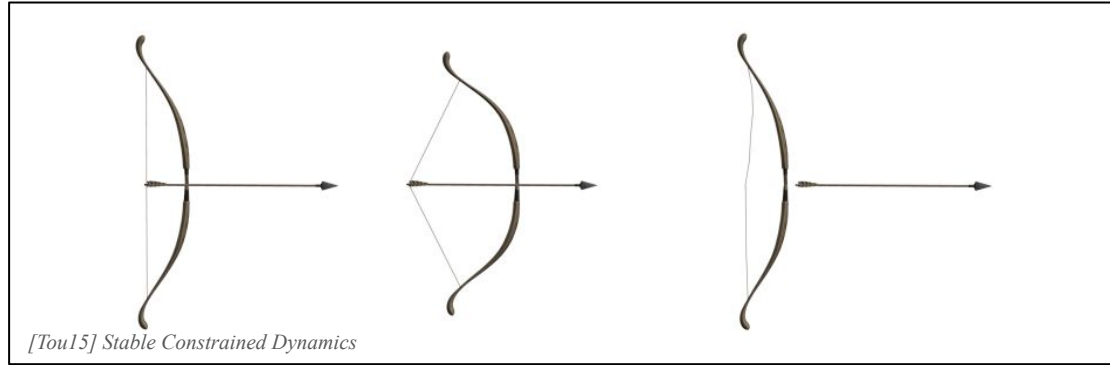


[Che24] - Vertex Block Descent

Élastique

Objets déformables élastiques

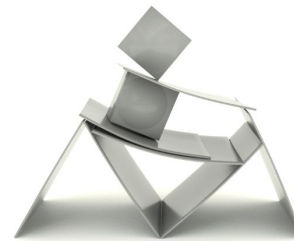
Objet capable de retrouver sa forme initiale après avoir été déformé par des interactions externes



[Zan19] - Scalable Galerkin Multigrid Method for Real-time Simulation of Deformable Objects



[Ang19] - VIPER: Volume Invariant Position-based Elastic Rods



[Li20] - Incremental Potential contact



[Zha22] - Progressive Simulation for Cloth Quasistatics

Cas d'applications temps réel

Temps de simulation = Temps dans la réalité

Jeux vidéos

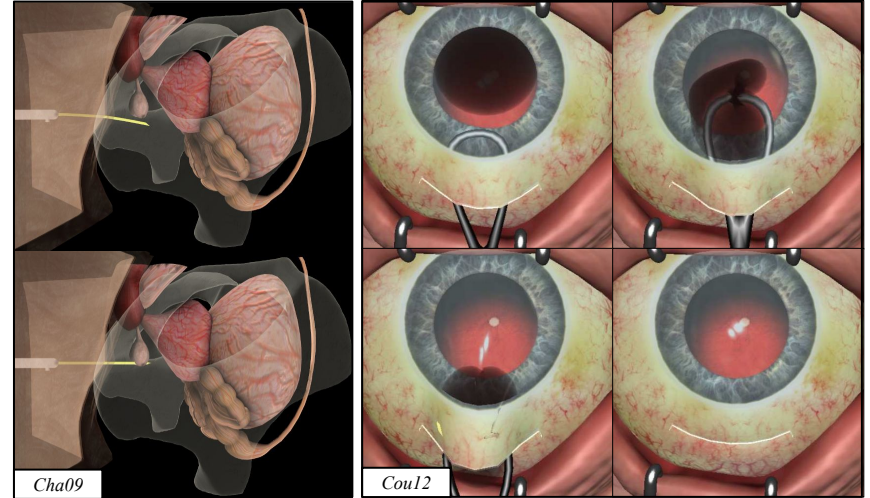


Temps de calcul par image ~ 16ms

Très robuste (non contrôlé)

Peu de précision/réalisme

Simulateur d'apprentissage médical



Rapide / Robuste / Précis

Matériaux organiques (quasi-incompressible)

Retour haptique (1000Hz)

Problématique

Objectifs

Objet déformable élastique

Temps réel (< 16 ms / image)

Quasi-incompressible

Numériquement robuste

Précision numérique élevée

Comment ?

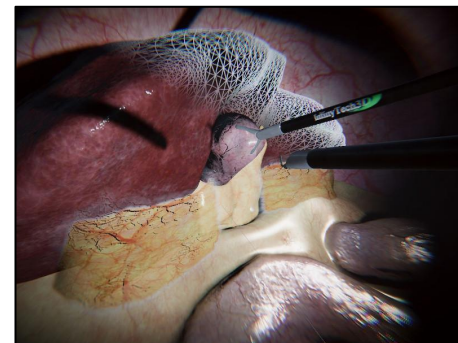
Éléments finis d'ordre élevé

Méthode de résolution locale

Optimisations spécifiques



Chaos Flesh (Unreal Engine 5)



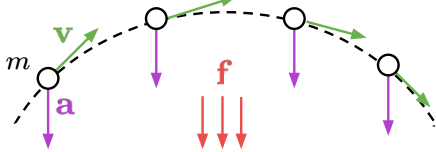
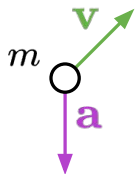
SOFA & Unity3D

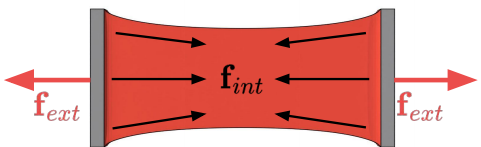
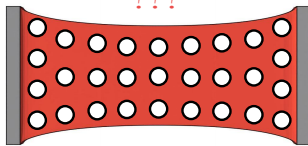


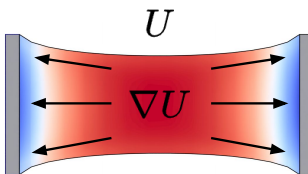
[Smi18] Stable Noe-Hookean Flesh Simulation

État de l'art : simulation d'objets déformables élastiques

Principe Fondamental de la Dynamique en élasticité

PFD	
$\mathbf{a} = m^{-1}\mathbf{f}$	
	
Discret	
	1 masse 1 position 1 vitesse 1 accélération n forces ...

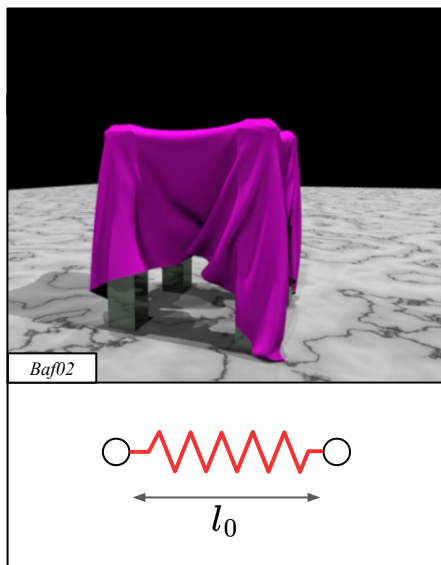
PFD (élasticité)	
$\mathbf{a} = m^{-1}(\mathbf{f}_{ext} + \mathbf{f}_{int})$	
	
Continu	
	

Forces internes élastiques	
Les forces ramènent l'objet à sa forme initiale	
$\mathbf{f}_{int} = -\nabla U$	
	
$U = \text{Déformation} \times \text{Matériau}$	

Quelle discrétisation utiliser ?

Modélisation d'un objet élastique

Masse-ressort



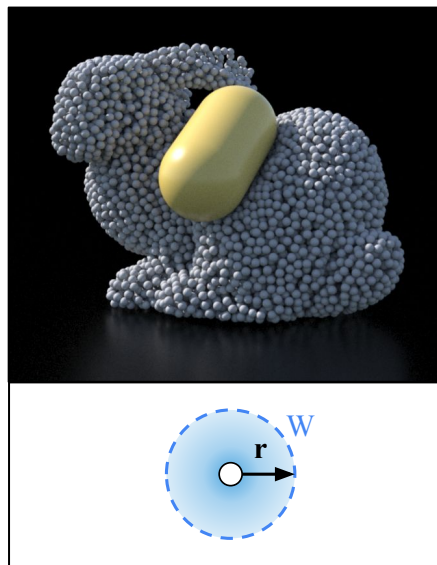
Simplicité de mise en oeuvre

Faible coût de calcul

Forte approximation

[Bad02] Robust Treatment of Collisions,
Contact and Friction for Cloth Animation

SPH



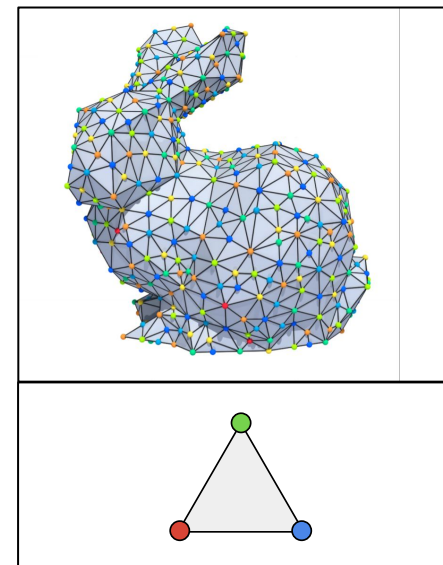
Forme dynamique

Gestion des collisions

Coût élevé
+ Artefact de déformation

[Pee18] An Implicit SPH Formulation for
Incompressible Linearly Elastic Solids

Éléments finis

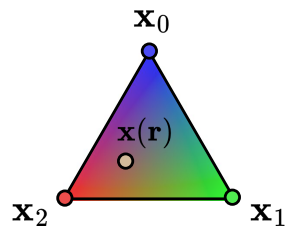


Maturité théorique

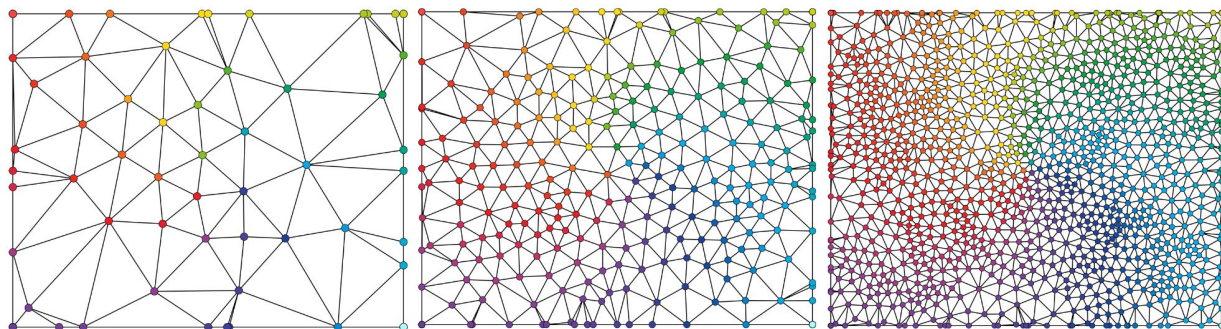
Fidélité physique / Validation

Coût de calcul

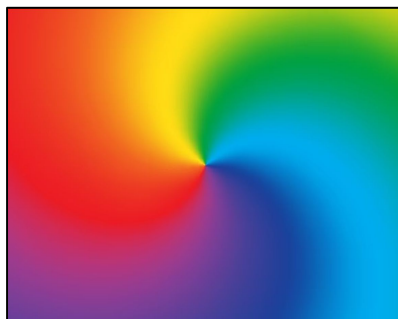
Discrétisation avec les éléments finis



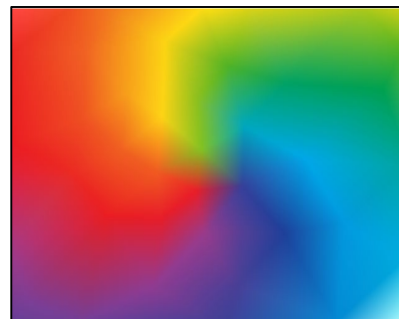
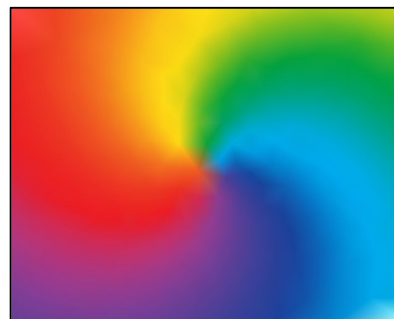
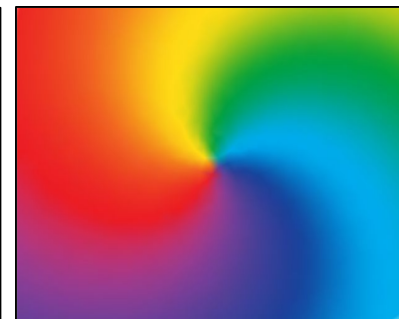
$$\mathbf{x}(\mathbf{r}) = \sum_i^n \mathbf{x}_i N_i(\mathbf{r})$$



Maillage



Référence

72 points
80 triangles264 points
433 triangles1032 points
1900 triangles

Approximation

Qualité

#Sommets

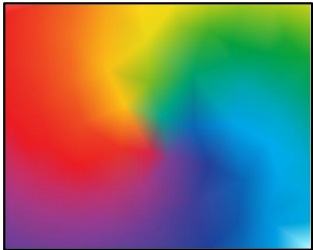
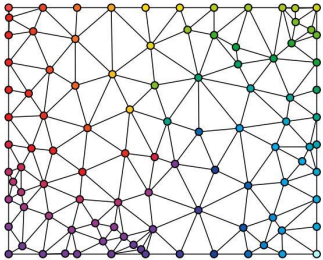
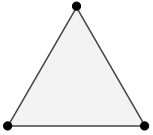
#Éléments

Type d'élément

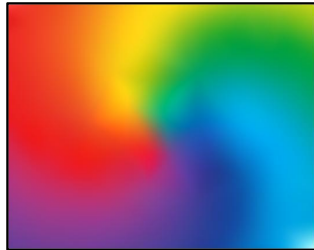
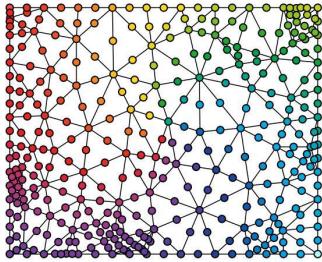
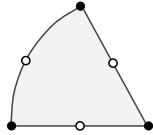
Ordre d'interpolation

Types et ordres des éléments

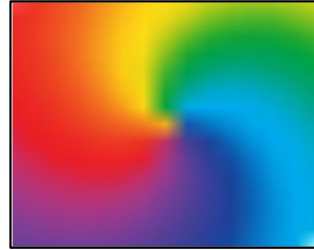
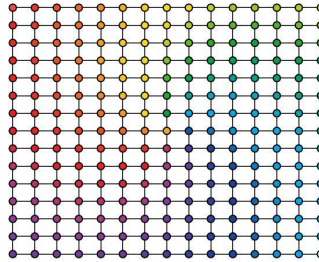
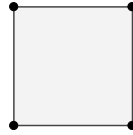
Triangle Linéaire



Triangle Quadratique



Quad Linéaire



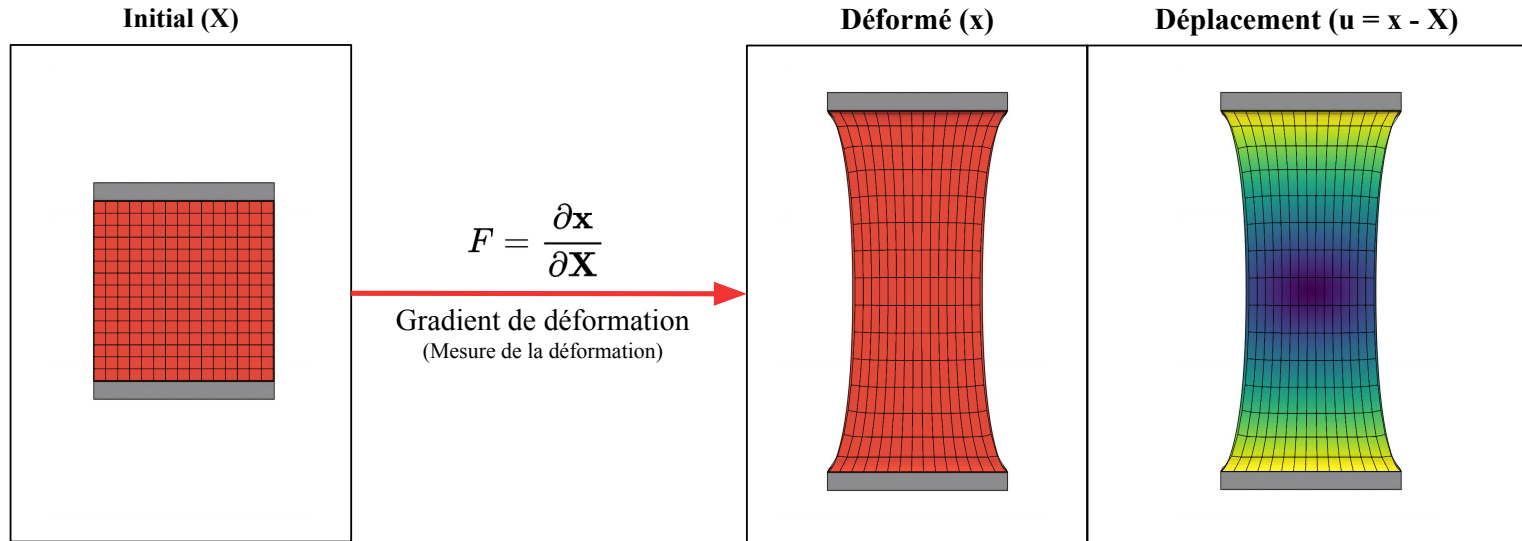
En 3D

		Tétraèdre	Hexaèdre
Linéaire	P1		
	Q1		
Quadratique	P2		
	Q2		

...

...

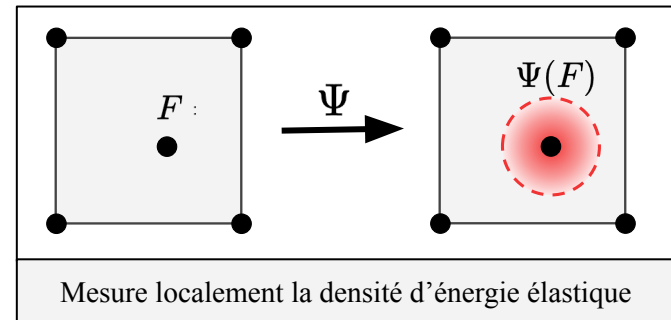
Déformation et énergie élastique



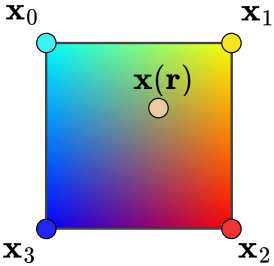
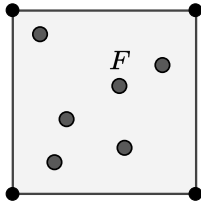
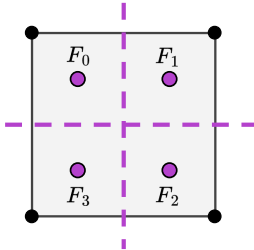
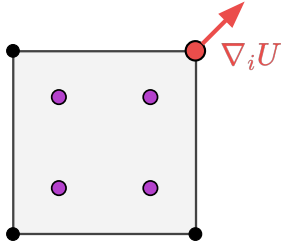
Lien avec la physique ?

Lois de comportements (Hooke, StVK, Neo-Hooke ...)

Déformation géométrique $\Rightarrow$ Densité d'énergie élastique



Résolution du *PFD* ?

1 - Élément fini	2 - Déformation	3 - Énergie élastique	4 - Gradient / Force
			
$\mathbf{x}(\mathbf{r}) = \sum_i^n \mathbf{x}_i N_i(\mathbf{r})$	$F = \frac{\partial \mathbf{x}}{\partial \mathbf{X}}$	$U = \sum_q^Q V_q \Psi(F_q)$	$\nabla_i U = \sum_q^Q V_q \frac{\partial \Psi(F_q)}{\partial \mathbf{x}_i}$
Approximation discrète	Mesure de la déformation	Évaluation de l'énergie avec les quadratures	Calcul des forces internes élastiques au sommet

Principe Fondamental de la Dynamique

$$\mathbf{a} = M^{-1}(\mathbf{f}_{ext} - \nabla U)$$

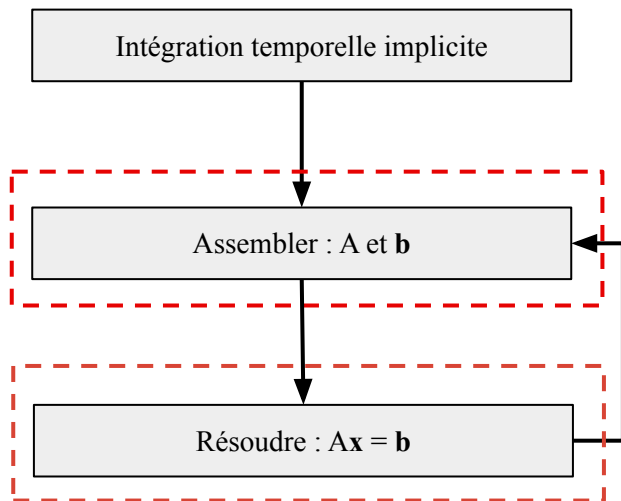
Quelle méthode utiliser pour résoudre le *PFD* avec les éléments finis ?

Efficacité

Robustesse

Méthodes de résolution locales pour le *PFD*

Une approche classique ?



A est la dérivée des forces (coût)

Résolution du système $\sim O(N^3)$

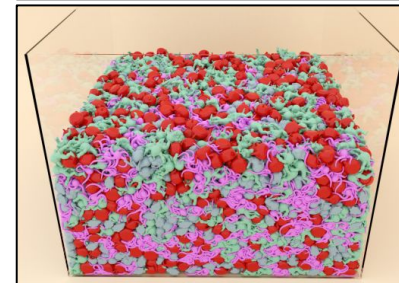
XPBD



[Mac19] - Small Steps in Physics Simulation

Approche par élément

Vertex Block Descent



[Che24] - Vertex Block Descent

Approche par sommet

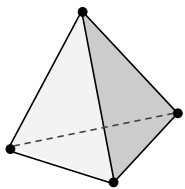
Matrix-Free + Temps réel + Robuste

mais uniquement définies pour les tétraèdres linéaires

Pourquoi ?

Pourquoi les tétraèdres linéaires ?

Tétraèdre linéaire

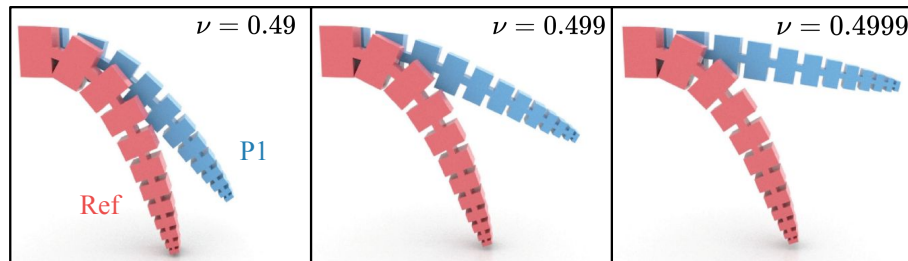


Simple / Coût faible

Précision faible par élément

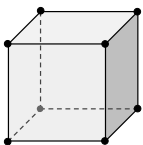
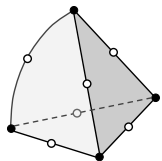
Artefact de *locking*

Matériaux quasi-incompressible $\Rightarrow$ raideur artificielle (*locking*)



[Frâ21] - Locking-Proof Tetrahedra

Autres types

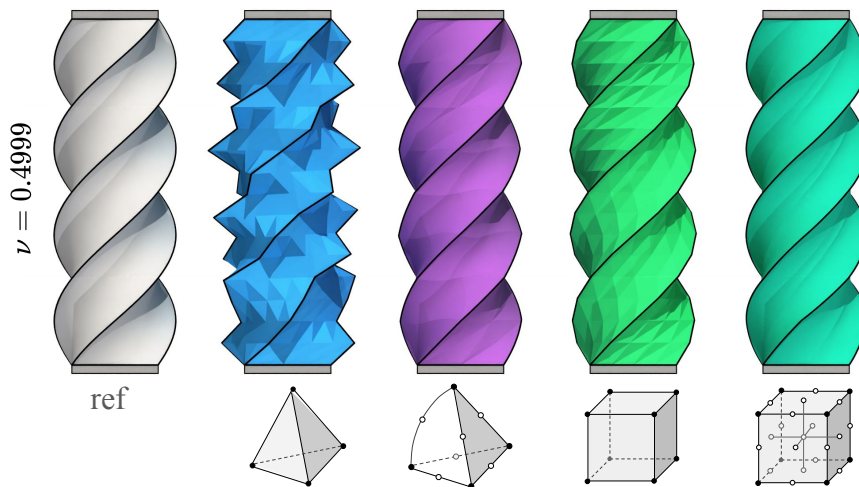


Pas de *locking*

Trop coûteux ?

Plus efficaces ? [Sch22]

Non vérifié pour le temps réel



Éléments d'ordre élevé avec *eXtended Position Based Dynamics*

Aperçu : *eXtended Position Based Dynamics*

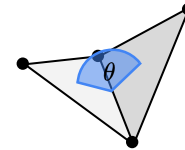
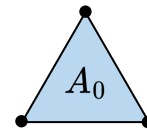
Énergies
en
Contraintes

$$U = \frac{1}{2} \alpha^{-1} C^2 \quad \alpha = k^{-1} \quad C = 0$$

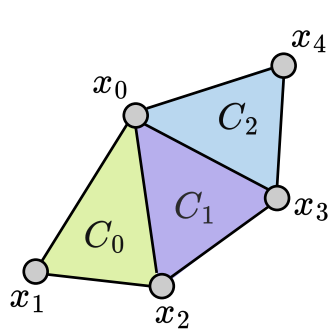
compliance

contrainte

Énergies $\Rightarrow$ Contraintes géométriques à respecter

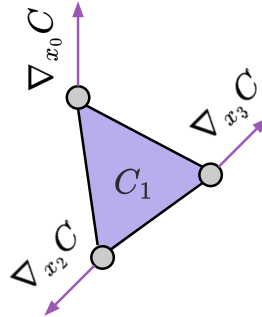


Partitionnement



Évite les conflits mémoire

Résolution



$$C_j \quad \nabla x_i C_j$$

Les points clefs

1 raideur par contrainte

Contrainte & son gradient

Partitionnement pour la parallélisation

Comment faire pour les éléments finis ?

Aperçu : éléments finis avec XPBD

[Mac21] - A Constraint-based Formulation of Stable Neo-Hookean Materials

Neo-Hooke Stable [Smith 2018]	$\Psi(F) = \underbrace{\left[\frac{\lambda}{2}\right] (det(F) - \gamma)^2}_{\text{Volume}} + \underbrace{\left[\frac{\mu}{2}\right] (tr(F^T F) - 3)}_{\text{Étirement}}$	
2 raideurs 2 contraintes	$\Psi(F) = \boxed{\Psi_\lambda(F)} + \boxed{\Psi_\mu(F)}$	
$U = \frac{1}{2} \boxed{V_{tet}} \Psi(F)$	$U_\lambda = \frac{\lambda}{2} \boxed{V_{tet}} (det(F) - \gamma)^2$	$U_\mu = \frac{\mu}{2} \boxed{V_{tet}} (tr(F^T F) - 3)$
Compliance volumique	$\alpha = (k V_{tet})^{-1}$	
$U = \frac{1}{2} \alpha^{-1} C^2$	$C_\lambda = det(F) - \gamma$	$C_\mu = \sqrt{tr(F^T F) - 3}$



Uniquement valide pour les tétraèdres linéaires

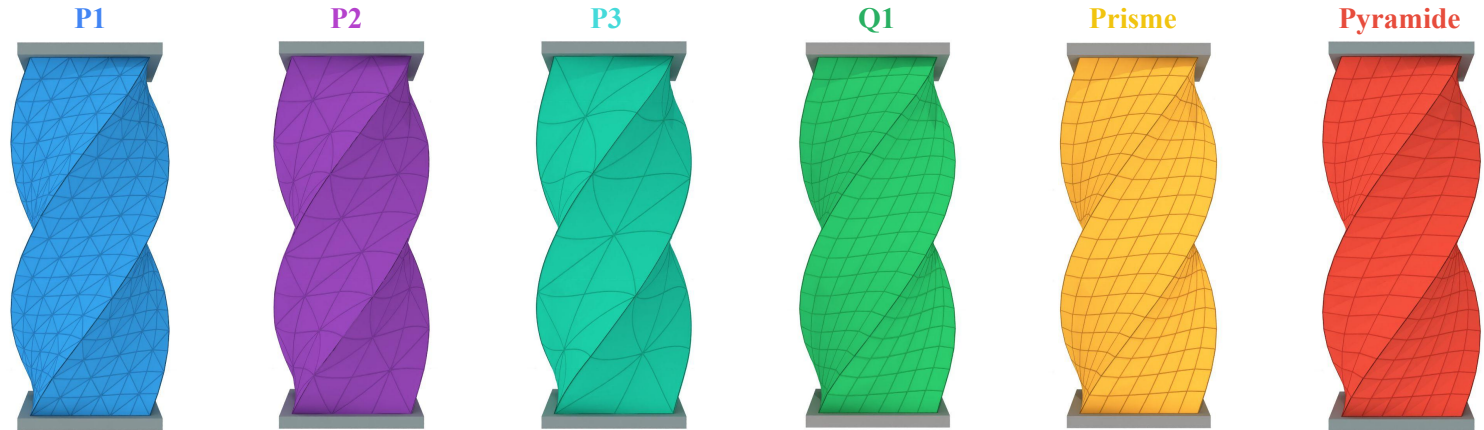
tétraèdres linéaires $\Rightarrow$ 1 évaluation x Volume

Construction des contraintes

corrections en l'absence de déformation

Contribution 1 : Contraintes pour tout type d'élément

	Énergie $\Leftrightarrow$ Contrainte	Conversion en contrainte	Gradient
XPBD	$U = \frac{1}{2} \alpha^{-1} C^2 \quad \alpha = k^{-1}$	$C = \tilde{U} ^{\frac{1}{2}} \quad \tilde{U} = 2\alpha U$ Enlève la raideur	$\nabla_i C = \frac{s}{2C} \nabla_i \tilde{U}$ $s = \tilde{U} / \tilde{U} $
FEM	$U = \sum_q^Q V_q \Psi(F_q)$	$C = \left[\left \sum_q^Q V_q \tilde{\Psi}(F_q) \right \right]^{1/2}$	$\nabla_i C = \frac{s}{2C} \sum_q^Q V_q \frac{\partial \tilde{\Psi}(F_q)}{\partial \mathbf{x}_i}$



Contribution 2 : Contraintes stables au repos

Est-ce qu'il y a des corrections lorsque l'objet n'est pas déformé ?

$$\tilde{\Psi}(F) = 0, \quad \frac{\partial \tilde{\Psi}(F)}{\partial F} = 0 \quad \text{avec} \quad F = I$$

Neo-Hooke Stable [Mac21]	
$\frac{\lambda}{2} (\det(F) - \boxed{\gamma})^2 + \frac{\mu}{2} (\text{tr}(F^T F) - 3) \boxed{\gamma = 1 + \frac{\mu}{\lambda}}$ $\downarrow$ $F = I$ $\downarrow$ $F = I$	
$\tilde{\Psi}_\lambda(I) = \boxed{\left(\frac{\mu}{\lambda}\right)^2}$	$\tilde{\Psi}_\mu(I) = 0$
$\frac{\partial \tilde{\Psi}_\lambda(I)}{\partial F} = \boxed{-2\frac{\mu}{\lambda} I}$	$\frac{\partial \tilde{\Psi}_\mu(I)}{\partial F} = \boxed{2I}$

instables

instables

Pourtant l'énergie est **stable** (forces nulles au repos) [Smith 2018]

Cela ne garantit pas des contraintes stables

Neo-Hooke Stable (développée)	
$\frac{\lambda}{2} (\det(F) - 1)^2 + \frac{\mu}{2} [\text{tr}(F^T F) - 2\det(F) - 1] + \cancel{\left(\frac{\mu}{\lambda}\right)^2}$ $\downarrow$ $F = I$ $\downarrow$ $F = I$	
$\tilde{\Psi}_\lambda(I) = 0$	$\tilde{\Psi}_\mu(I) = 0$
$\frac{\partial \tilde{\Psi}_\lambda(I)}{\partial F} = 0$	$\frac{\partial \tilde{\Psi}_\mu(I)}{\partial F} = 0$

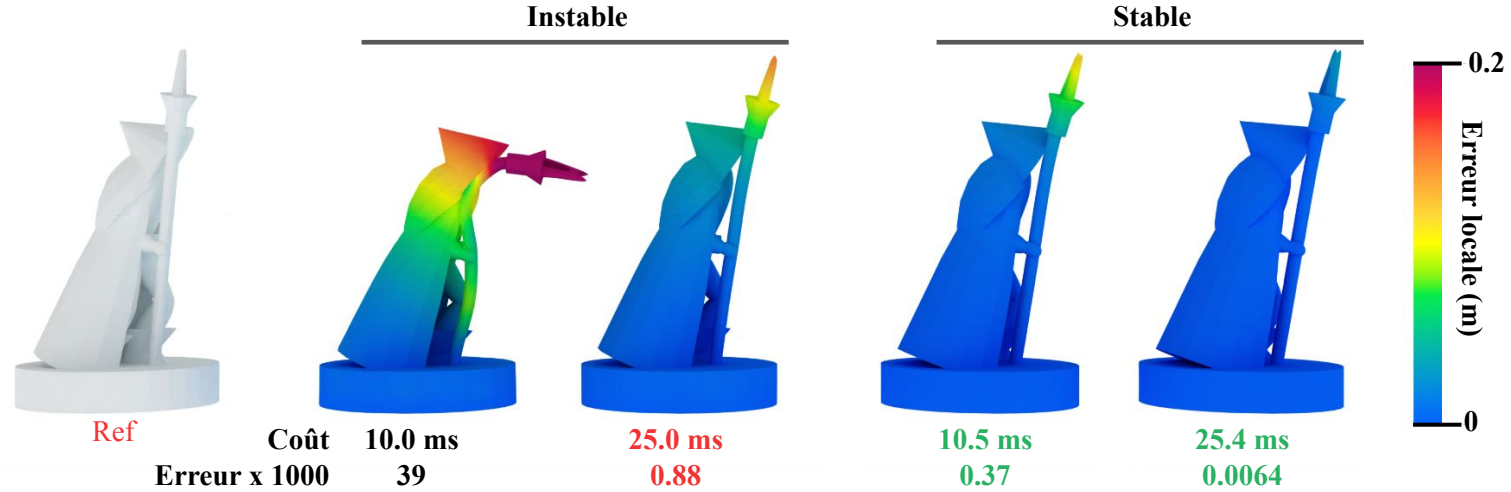
stables

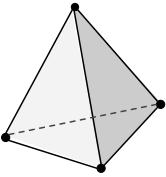
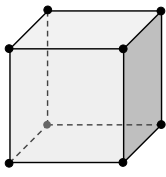
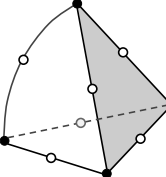
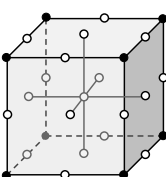
stables

La **correction** est appliquée sur le bon terme

Impact la résolution des contraintes ?

Expérience 1 : Gain des contraintes stables



Coût à convergence [test de flexion]			
			
x3.9	x4	x4	x3.2

Gain en temps

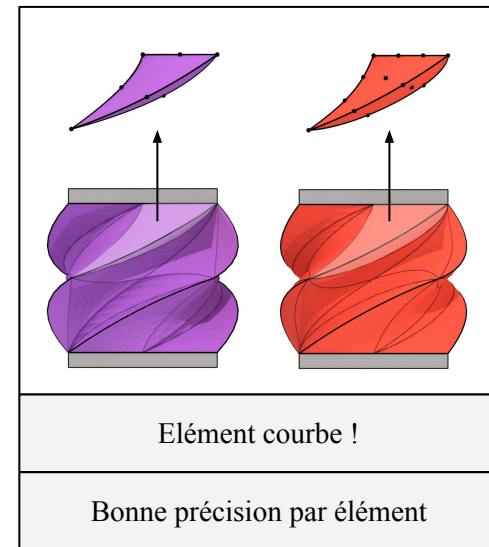
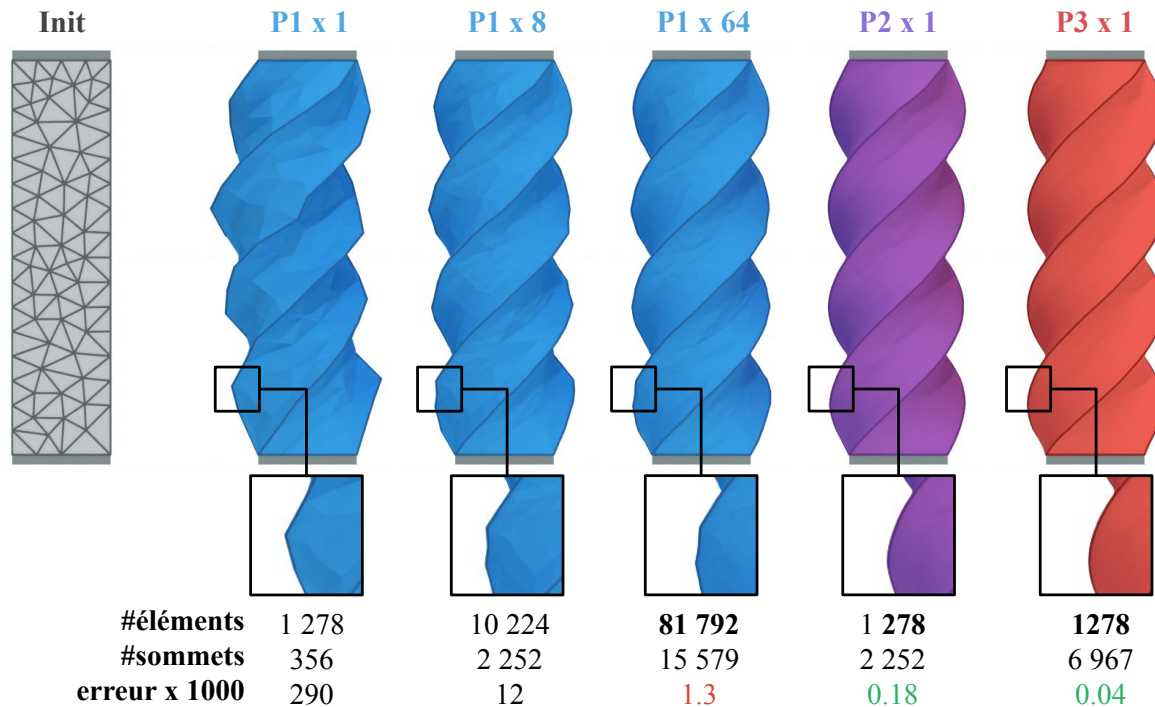
Meilleure précision pour un coût

Pour les autres éléments ?

Gain de x3 à x4

Est-ce que les autres éléments sont attractifs ?

Expérience 2 : Précision des ordres élevés

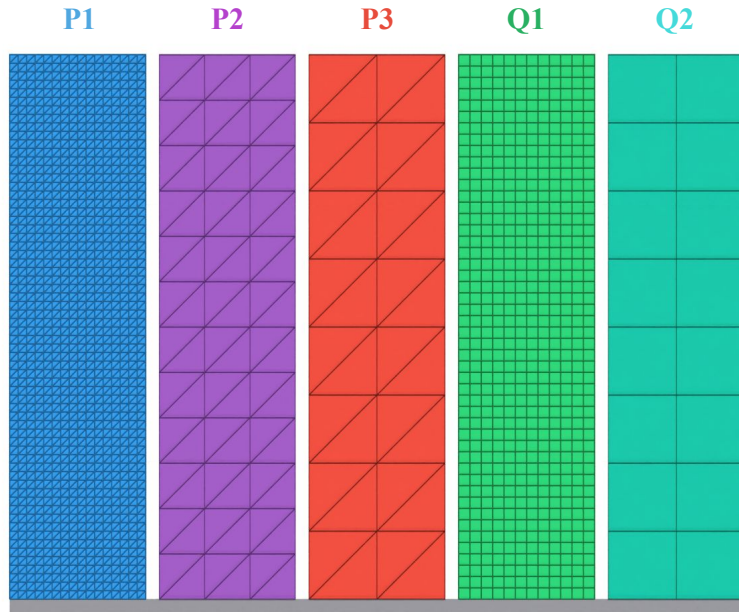


Ratio coût / précision des éléments ?

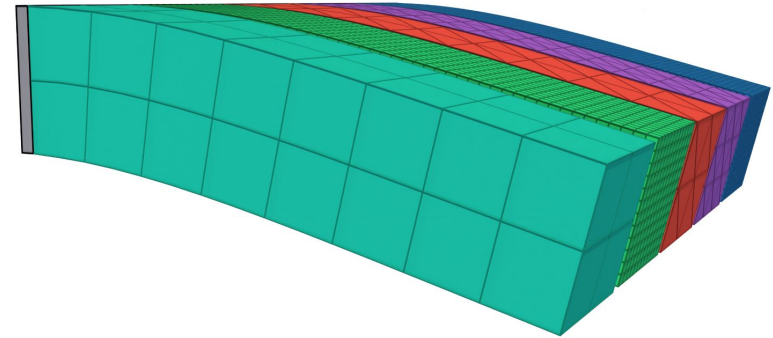
64 fois moins d'éléments

Pour une précision supérieure

Expérience 3 : Test de flexion



Comparaison pour une erreur similaire



Faible conservation de volume $\Rightarrow$ pas de locking

Comparaison équitale avec P1

Expérience 3 : Coût à précision équivalente

Implémentation GPU (partitionnement des contraintes)

Test de flexion				
Type	#Éléments	#Sommets	GPU	Erreur x 10 ⁵
P1	81 920	18 785	14 ms	63
P2	540	1 117	16.5 ms	5.7
P3	160	1 097	47 ms	6.4
Q1	3 456	8 281	6.3 ms	3.1
Q2	32	425	28.3 ms	2.6

Gain ou équivalent (Q1 / P2)

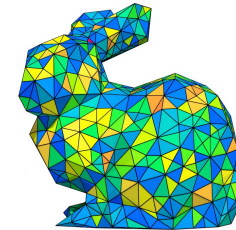
Meilleure coloration des hexaèdres

P1 n'arrive pas à convergence ?

Les autres aussi !

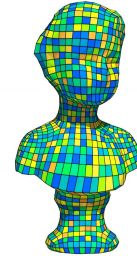
Partitionnement pour la parallélisation

Tétraèdres



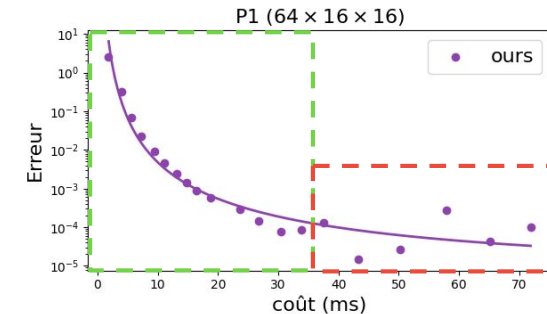
48 Partitions

Hexaèdres



12 Partitions

Convergence limitée avec XPBD



Conclusion : Apports et limitations pour *XPBD*

Publication 2024

Contraintes stables

Gain pour tous les éléments

Élément d'ordre élevé en *XPBD*

Gain ou équivalent sur GPU

Conservation de volume

Limitations

Limité en précision
Pas de maillages denses !

Mauvaise parallélisation *GPU*

Autres approches

Plus adaptée pour le *GPU* ?

CGI2024 manuscript No.
(will be inserted by the editor)

High-Order Elements in Position Based Dynamics

Bastien Sallant* · Florence Zara · Guillaume Damiani · Fabrice Jaillet

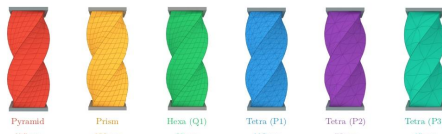


Fig. 1: 180-degree torsion of beams composed of different types of elements: pyramids, prisms, hexahedra and tetrahedra, considering linear (P1), quadratic (P2) and cubic (P3) tetrahedra. The timing indicates the average computation time for each XPBD simulation loop.

Abstract The simulation of deformable objects has been the subject of a great deal of work in the field of computer graphics. The constraint-based PBD (Position Based Dynamics) approach has been proven to be effective in this field for real-time and stable deformable objects simulation. Finite Element Method with linear tetrahedron discretization are the most widely used in Computer Graphics despite producing less accurate results than hexahedral or higher order elements. In this context, our proposal is to integrate higher degree elements within the PBD framework. In addition, we propose a solution to improve convergence of unstable energies (like Neo-Hooke) when used as constraints. We show that our approach improves accuracy compared to linear tetrahedra. We also highlight the time savings, since fewer elements are needed.

B. Sallant
E-mail: bastien.sallant@iris.cnr.fr, INSA Lyon, UCBL, CNRS, LIRIS, UMR5205, F-69622 Villeurbanne
F. Zara
E-mail: florence.zara@iris.cnr.fr, UCBL, CNRS, INSA Lyon, LIRIS, UMR5205, F-69622 Villeurbanne
F. Jaillet
E-mail: fabrice.jaillet@iris.cnr.fr, IUT Lyon 1, UCBL, CNRS, INSA Lyon, LIRIS, UMR5205, F-69622 Villeurbanne
G. Damiani
E-mail: guillaume.damiani@iris.cnr.fr, CNRS, INSA Lyon, UCBL, LIRIS, UMR5205, F-69622 Villeurbanne

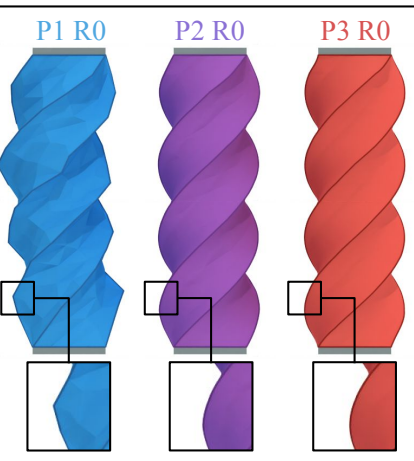
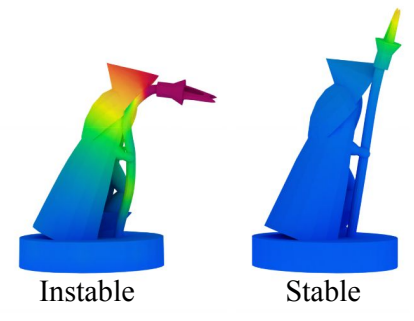
Keywords Physical Based Simulation · Position Based Dynamics · High-order elements · Elasticity · Realtime Physics.

1 Introduction

In Computer Graphics, a lot of effort have been invested in the field of simulation of deformable objects. One of the challenges has been to develop physically-based models that are sufficiently accurate to be used in medical applications [31,29,11,10], while also offering real-time interaction with dynamic simulation.

For this purpose, well-known physical models have been adapted to achieve real time performances. For example, adaptive Finite Element methods [11,29,11] have been proposed, or the realism of discrete mass-spring model [28] has been improved by linking the springs' mechanical parameters to material properties such as Young's modulus or Poisson's ratio [32,17]. The interested reader can refer to these surveys [20,19,3] for a better understanding of these physical models for dynamic simulation.

More recently, the position-based dynamics (pbd) approach [2,3] has become popular. Due to its simplicity and robustness, many physical models have been defined within this framework [4]. To bring this approach closer to physics, the extension of pbd, called XPBD [23], provides a direct correlation with elastic energy and dissipation potentials. An adaptation has also been proposed incorporating a stable Neo-Hooke en-



Parallélisation des ordres élevés sur *GPU*
avec
Vertex Block Descent

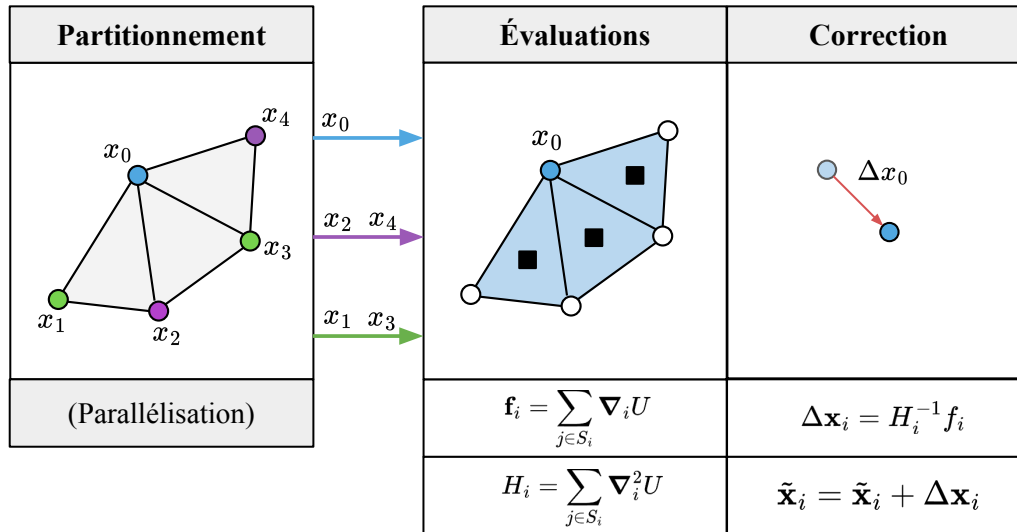
Aperçu : Résolution par sommet avec VBD

VBD

$$\arg \max_{\mathbf{x}_i} G_i(\mathbf{x})$$

PFD $\Rightarrow$ minimisation d'énergie par sommet

$$G_i(\mathbf{x}) = \underbrace{\frac{m_i}{2h^2} \|\mathbf{x}_i - \mathbf{y}_i\|^2}_{\text{inertie}} + \underbrace{\sum_{j \in S_i} U_j}_{\text{élasticité}} + \dots$$



Les points clefs

énergie dépend des éléments voisins

force + hessienne au sommet

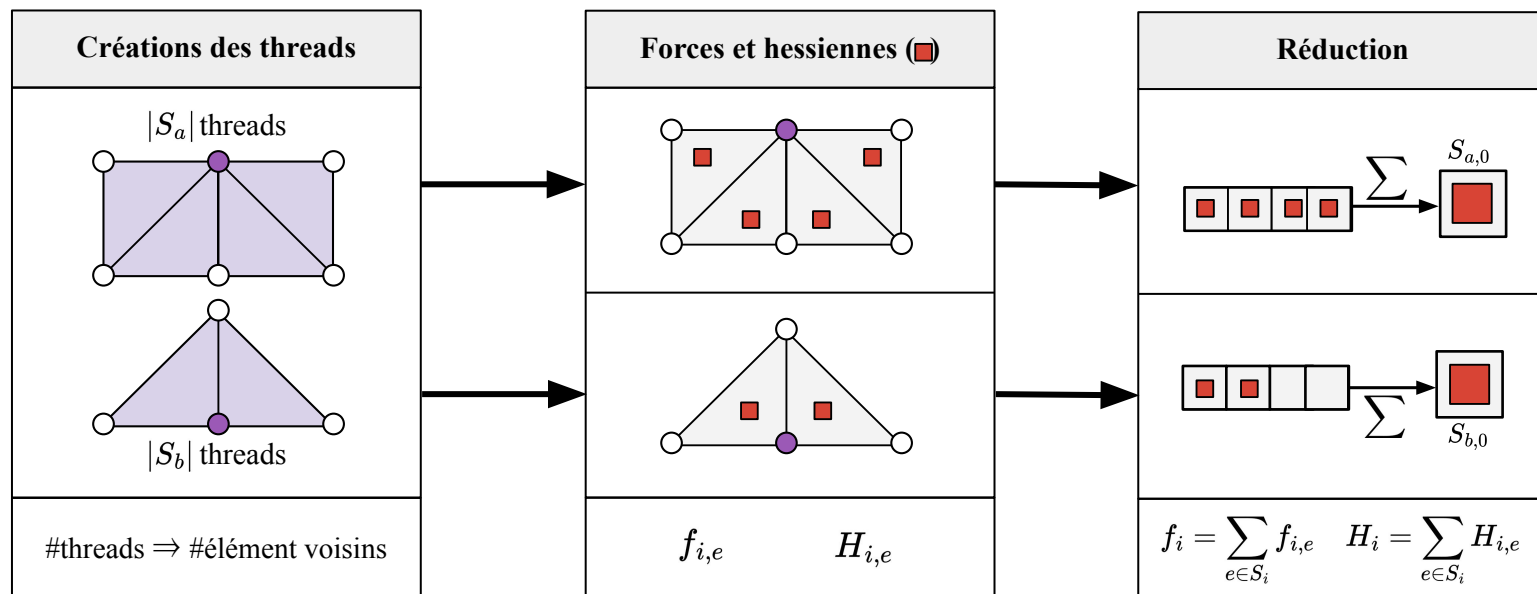
résolution directe du système local

Partitionnement (en général) efficace

Mais pas que !

Méthode adaptée au GPU

Aperçu : Parallélisation des calculs sur *GPU* avec VBD



Évaluations
$f_i = \sum_{e \in S_i} \nabla_i U_e \quad H_i = \sum_{e \in S_i} \nabla_i^2 U_e$

Des étapes rapides sur *GPU*

1 ordre de grandeur d'accélération des calculs

Exploiter ce mécanisme pour les autres éléments ?

Contribution : Parallélisation des évaluations sur GPU

Évaluations tous éléments

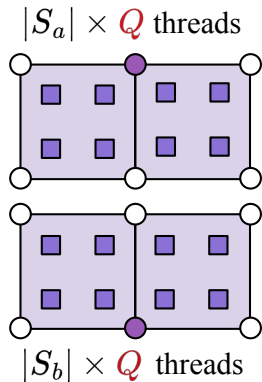
$$f_i = \sum_{e \in S_i} \sum_q^Q \nabla_i U_{e,q} \quad H_i = \sum_{e \in S_i} \sum_q^Q \nabla_i^2 U_{e,q}$$

Formulation locale des éléments finis

Voisinage $\times$ le nombre d'évaluations

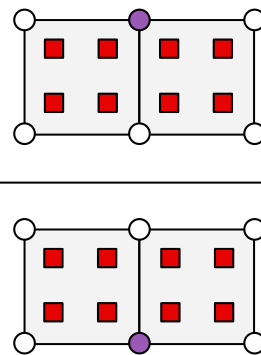
Tout peut se faire en parallèle !

Créations des threads



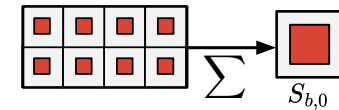
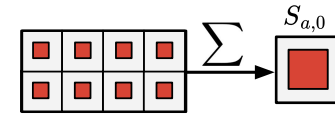
#threads $\Rightarrow$ #éléments voisins

Forces et hessiennes (■)



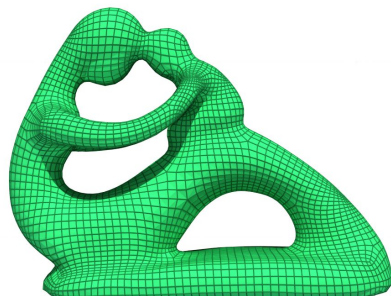
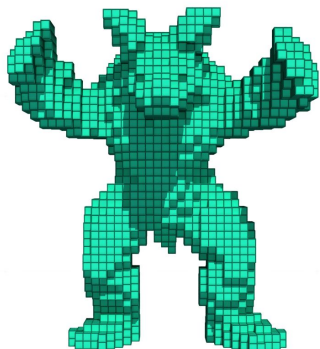
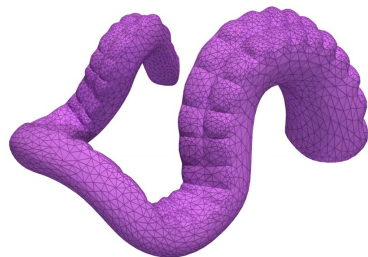
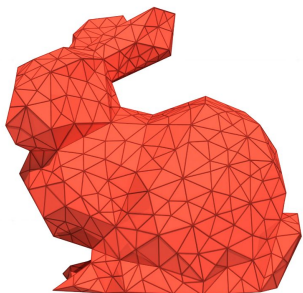
$f_{i,e,q} \quad H_{i,e,q}$

Réduction



$$f_i = \sum_{(e,q)}^{S_i \times Q} f_{i,e,q} \quad H_i = \sum_{(e,q)}^{S_i \times Q} H_{i,e,q}$$

Expérience 1 : Accélération de la parallélisation des quadratures



#threads			
Type	Maillage	$ S_i \times Q$	Gain
Q1	Fertilité	$10 \times 8 = 80$	$\times 1.8$
	Lapin	$10 \times 8 = 80$	$\times 1.9$
P2	Miniature	$56 \times 4 = 224$	$\times 1.5$
	Tentacule	$68 \times 4 = 272$	$\times 1.4$
Q2	Armadillo R4	$12 \times 27 = 324$	$\times 2.8$
	Armadillo R5	$12 \times 27 = 324$	$\times 3.5$
P3	Cube 8^3	$32 \times 11 = 352$	$\times 2.1$
	Cube 12^3	$33 \times 11 = 363$	$\times 2.3$
	Lapin <i>low poly</i>	$45 \times 11 = 495$	$\times 1.0$

Meilleure parallélisation GPU

mais

Limitée par la mémoire partagée

Expérience 2 : Coût à précision équivalente

Précision équivalente, pas de *locking*, etc.

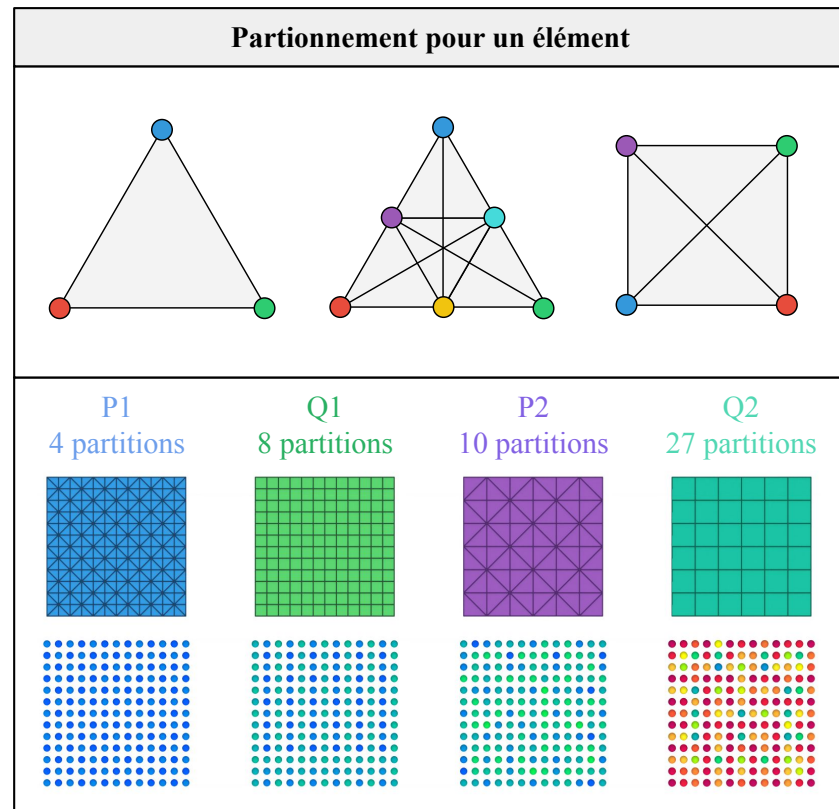
Test de flexion					
Type	#Éléments	#Sommets	Coût	Gain	Erreur x 10 ⁵
P1	98 260	22 356	9.5 ms	× 1	5.0
P2	540	1 117	5.5 ms	× 1.7	3.8
Q1	4 000	4 961	4.3 ms	× 2.2	5.3
Q2	108	1 225	9.8 ms	× 0.97	1.6

Q1 et P2 environs deux fois plus rapide

Mauvaise parallélisation de Q2 ?

Le partitionnement dépend fortement de la topologie des éléments

(Mais on est temps réel)



Conclusion VBD : Apports et limitations

Publication 2025

Stratégie des sous-étapes

Gain en performance (x5 à x10)

Élément d'ordre élevé en VBD

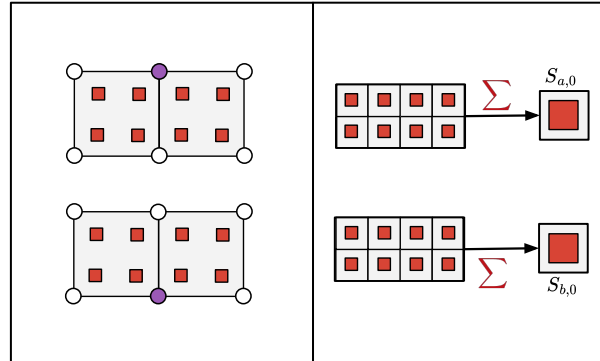
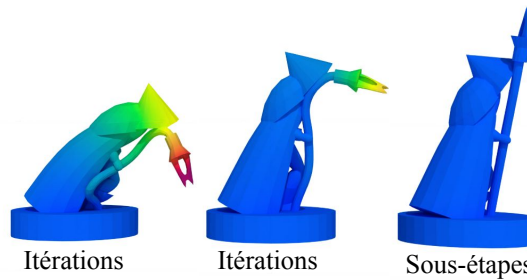
Parallélisation des éléments

Gain de x2 contre P1

Problème parallélisation GPU ?

c'est mieux mais ...

Pourquoi les éléments d'ordre élevé sont mal parallélisés ?



Going further with Vertex Block Descent

B. Saillant
INSA Lyon, UCBL, CNRS, LIRIS, UMR5205, F-69622 Villeurbanne
bastien.sailant@liris.cnrs.fr

[Website](#)

F. Zara
UCBL, CNRS, INSA Lyon, LIRIS, UMR5205, F-69622 Villeurbanne
florence.zara@liris.cnrs.fr

[Website](#)

F. Jaillot
UCBL, CNRS, INSA Lyon, LIRIS, UMR5205, F-69622 Villeurbanne
fabrice.jaillot@liris.cnrs.fr

[Website](#)

G. Damiand
CNRS, INSA Lyon, UCBL, LIRIS, UMR5205, F-69622 Villeurbanne
guillaume.damiand@liris.cnrs.fr

[Website](#)



Figure 1: Simulation with 46,944 P_3 elements using our extension of VBD for high order elements.

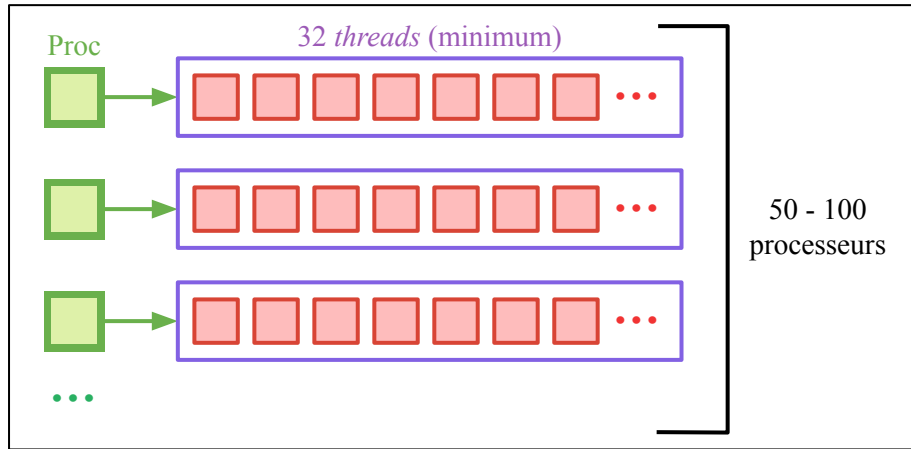
Abstract

Vertex Block Descent (VBD) is a fast and robust method for the real-time simulation of deformable objects using the finite element method. Originally, the method is designed for the popular linear tetrahedral elements. However, these elements have low accuracy and introduce locking artifacts. In this context, we propose an extension of VBD in order to: (i) support any isoparametric elements and shape function orders; (ii) improve its performance even for high-order elements; and (iii) enhance convergence through the use of sub-stepping. Overall, using other types of elements enables more accurate results while maintaining a computational cost comparable to linear tetrahedra.

Keywords: Vertex Block Descent (VBD), real-time simulation, FEM, high-order elements, GPU, physically-based simulation.

Saturation du *GPU* pour les ordres élevés avec *XPBD* et *VBD*

Problématique : Maximiser l'utilisation du GPU



Saturation du GPU

32 threads par processeur

Plusieurs dizaines de processeurs

Besoin de plusieurs **milliers** de threads

Type	#éléments	#partitions	#threads	#proc	Coût = #Q
P1	81 920	32	2560	80	1
P2	540	32	17	1	4
P3	160	32	5	1	11
Q1	3 456	8	432	14	8
Q2	32	8	4	1	27

Avec XPBD en ordre élevé ?

$\#threads = \#éléments / \#partitions$

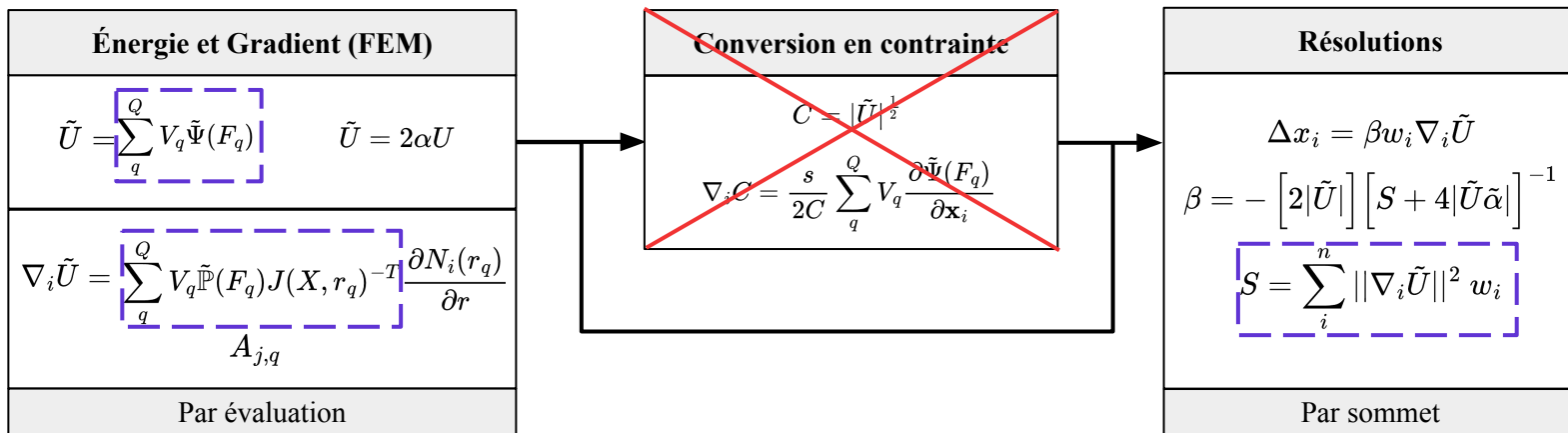
$\#proc = \#threads / 32$

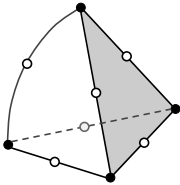
Pas assez de threads !

Meilleur partitionnement ? **Déjà optimal ...**

Parallélisation des étapes de calculs ?

Contribution : Parallélisation de la résolution pour XPBD



 10 sommets 4 évaluations	0	$\tilde{U}_{j,0}$	$A_{j,0}$	Réduction $\sum_q^Q \tilde{U}_{j,q}$	S_0	Réduction $\sum_i^n S_i$	$\bar{x}_0 \leftarrow \bar{x}_0 + \Delta x$	
	1	$\tilde{U}_{j,1}$	$A_{j,1}$		S_1		$\bar{x}_1 \leftarrow \bar{x}_1 + \Delta x$	
	2	$\tilde{U}_{j,2}$	$A_{j,2}$		S_2		$\bar{x}_2 \leftarrow \bar{x}_2 + \Delta x$	
	3	$\tilde{U}_{j,3}$	$A_{j,3}$		S_3		$\bar{x}_3 \leftarrow \bar{x}_3 + \Delta x$	
	4	Inactifs					S_4	$\bar{x}_4 \leftarrow \bar{x}_4 + \Delta x$
	5						S_5	$\bar{x}_5 \leftarrow \bar{x}_5 + \Delta x$
	6						S_6	$\bar{x}_6 \leftarrow \bar{x}_6 + \Delta x$
	7						S_7	$\bar{x}_7 \leftarrow \bar{x}_7 + \Delta x$
	8						S_8	$\bar{x}_8 \leftarrow \bar{x}_8 + \Delta x$
	9						S_9	$\bar{x}_9 \leftarrow \bar{x}_9 + \Delta x$

Calcul direct de la correction

Parallélisation par sommet

threads inactifs au début si
#Quadratures $\neq$ #Sommets

inactifs de toute façon !

Expérience : Gain de l'accélération de la résolution pour *XPBD*

Test de flexion								
Type	#N	#M	Coût B_{ase}	Coût O_{pti}	B_{ase}/O_{pti}	$P1/O_{pti}$		
P1	18785	81920	14 ms			$\times 1$		
P2	1117	540	16.7 ms	7.6 ms	$\times 2.2$	$\times 1.8$		
P3	1097	160	47.0 ms	11.0 ms	$\times 4.3$	$\times 1.3$		
Q1	8281	3456	6.3 ms	4.6 ms	$\times 1.4$	$\times 3$		
Q2	425	32	28.3 ms	3.4 ms	$\times 8.3$	$\times 4.1$		

Accélération de 1.4 à 8 avec parallélisation

Autres éléments 1.5 à 4 fois plus rapides que P1

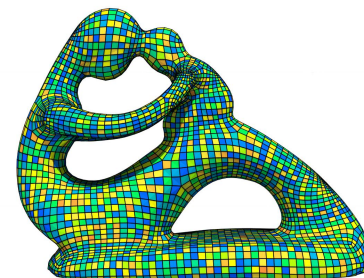
La coloration limite toujours les tétraèdres

Temps réel !

Coloration des tétraèdres

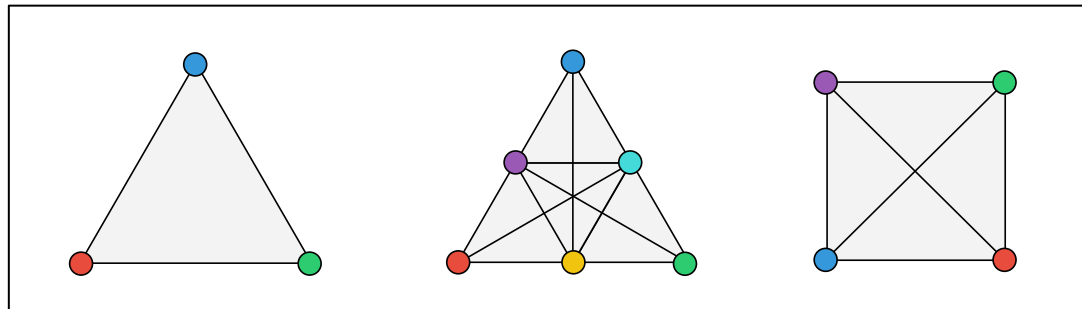


44 partitions



12 partitions

Problématique : Saturation du GPU pour *Vertex Block Descent*



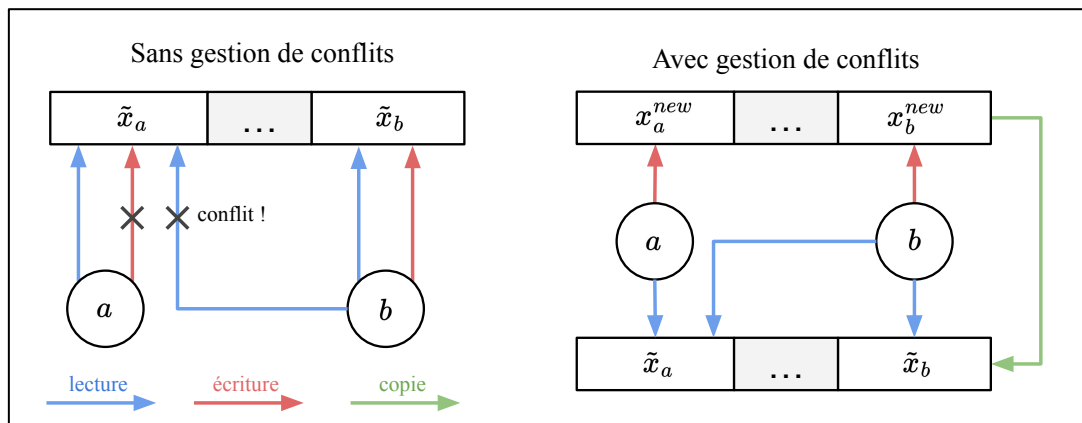
Problème de saturation avec *VBD*

Partitionnement limitant

$\#partitions \geq \#sommets$ d'un élément

Déjà optimal ou proche ...

Mais !



Gestion de conflits mémoire

Séparation lecture / écriture [Che24]

Nécessite une phase de copie

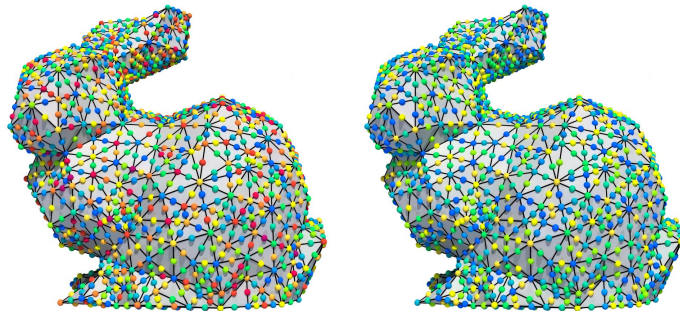
Permet des partitionnements avec conflits

Déjà utilisé pour les collisions

Mais pas pour les éléments

Contribution 2 : Partitionnement réduit pour les éléments finis

Réduction des partitions avec P2



#Partitions = 13

#Partitions = 8

#partitions devient arbitraire

#partitions trop faible $\Rightarrow$ instable

Impacte la convergence du solveur

Est-ce que c'est vraiment rentable ?

Test de robustesse (étirement) en fonction du #Partitions

#Partitions	1	2	3	4
P1				
Q1				
P2				
Q2				

Expérience 2 : Gain d'une coloration réduite pour VBD

Précision équivalente, pas de *locking*, etc.

Test de flexion				
Element	Coût B_{ase}	Coût O_{pti}	B_{ase} / O_{pti}	P_1 / O_{pti}
P1	9.5 ms	9.1 ms	$\times 1.04$	$\times 1.0$
P2	5.5 ms	3.1 ms	$\times 1.8$	$\times 2.9$
Q1	4.3 ms	2.2 ms	$\times 2.0$	$\times 4.1$
Q2	9.8 ms	3.3 ms	$\times 3.0$	$\times 2.8$

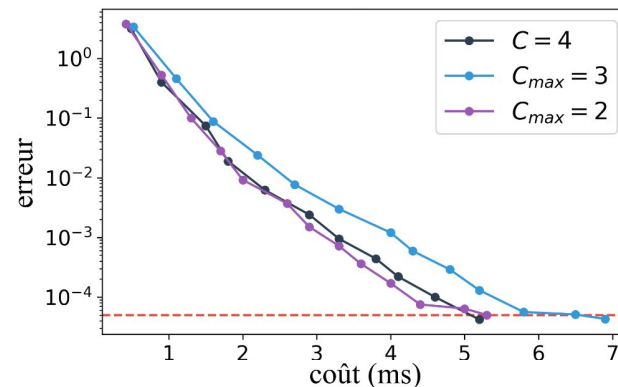
	P1	Q1	P2	Q2
C_{max}	2	2	3	4

Accélération de 2 à 3 (sauf P1)

Autres éléments 3 à 4 fois plus rapides que P1

Largement temps réel !

Convergence des tétraèdres linéaires

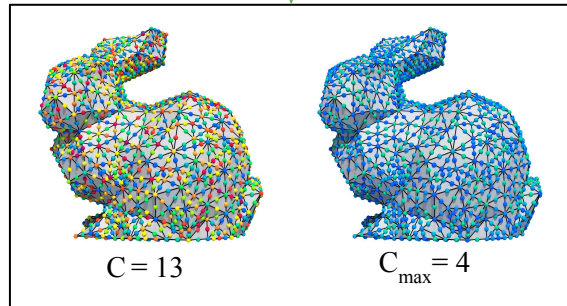
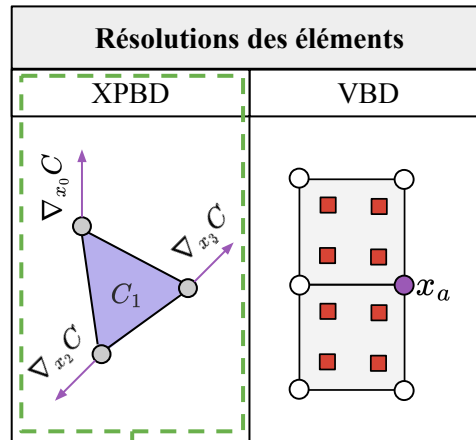
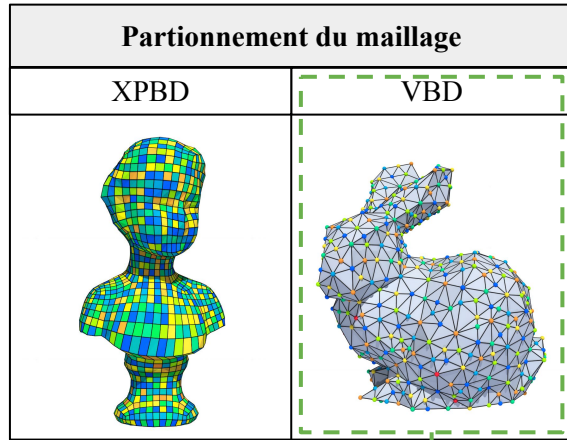


Partionnement efficace $\Rightarrow$ GPU saturé

Coût de la gestion de conflit

Réduction des partitions non rentable
pour les tétraèdres linéaires

Conclusion : Saturation du *GPU*



$$\Delta x_i = \beta w_i \nabla_i \tilde{U}$$

$$\beta = - \left[2|\tilde{U}| \right] \left[S + 4|\tilde{U}\tilde{\alpha}| \right]^{-1}$$

$$S = \sum_i^n ||\nabla_i \tilde{U}||^2 w_i$$

Temps réel
(possible d'améliorer les performance)

Optimisations matérielles

Réduction améliorée
Mémoire partagée réduite par 32

Patterns d'accès mémoire
accès coalescé / adresses continues

Chargement Coopératif
éviter les accès mémoire redondants

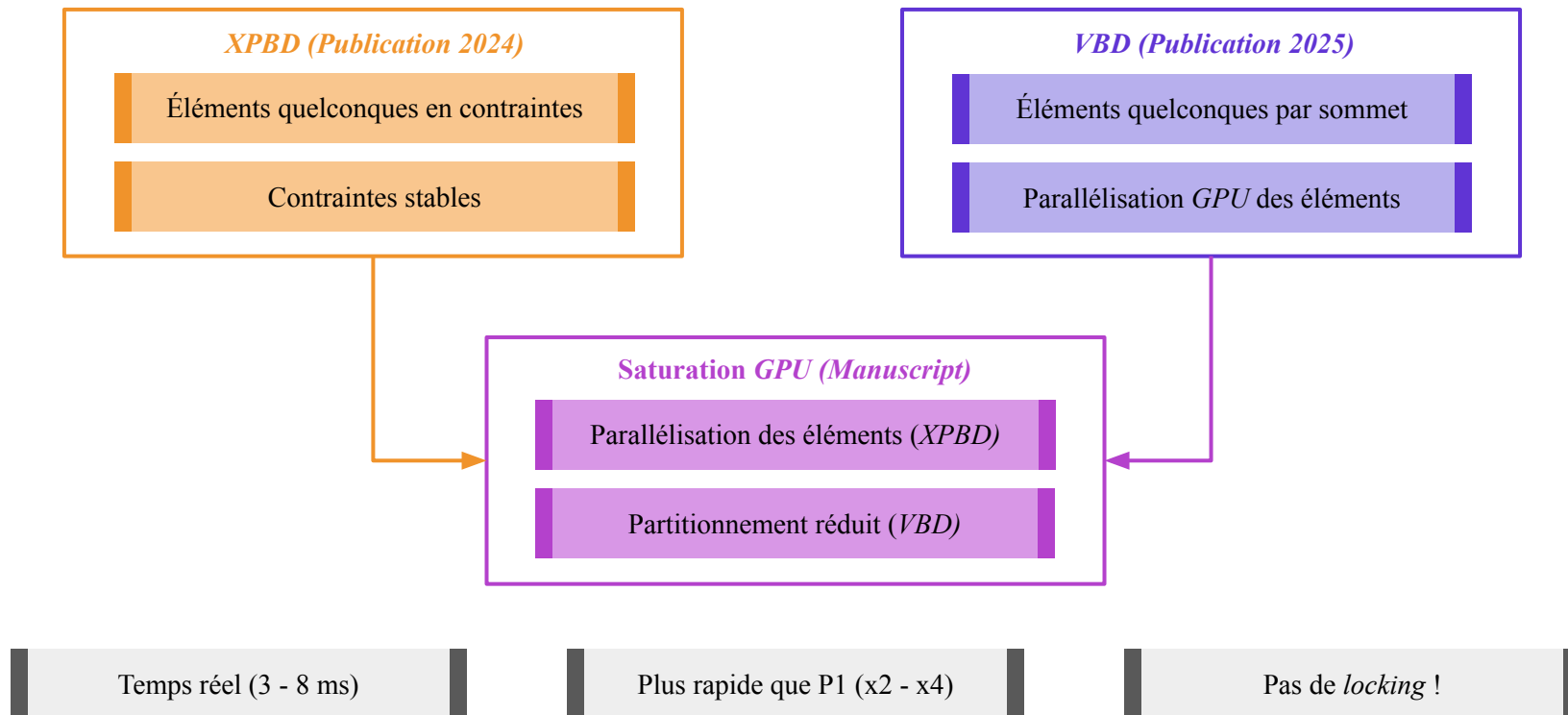
Spécialisation de code

...

Conclusion et Perspectives

Conclusion

Est-ce que d'autres éléments sont viables pour la simulation en temps réel ?



A priori, c'est rentable. Mais, nuançons un peu.

Les limitations et perspectives

Tests mécaniques simples

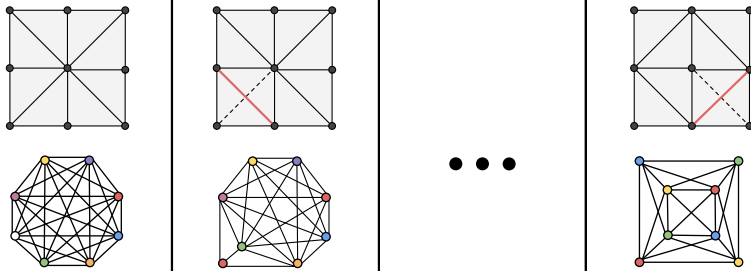
Demande plus d'éléments en pratique

Convergence des méthodes locales

Limitées à des maillages avec peu d'éléments

Quelques idées pour la suite (Moyen terme)

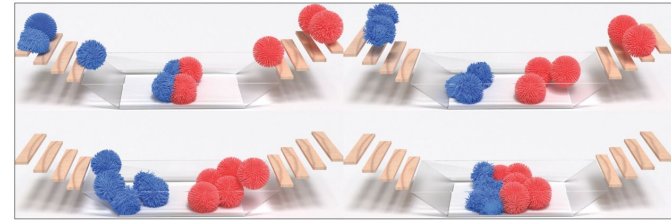
Optimisation de la topologie pour le partitionnement



Amélioration de la parallélisation (notamment *XPBD*)

Méthode de résolution locale-globale [2025]

JGS2: Near Second-order Converging Jacobi/Gauss-Seidel for GPU Elastodynamics



Adapter la méthode pour tout élément + améliorer la réduction

Pour aller encore plus loin

r-adaptation temps réel

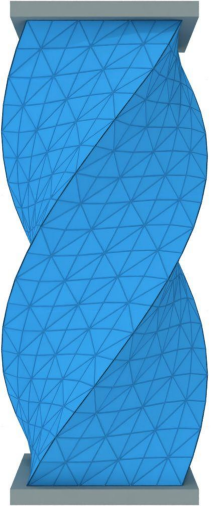
Adaptation par réallocation de la géométrie

Augmented Vertex Block Descent [2025]

Incompressibilité pour tous les éléments

Questions

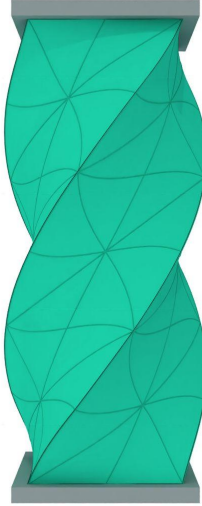
P1



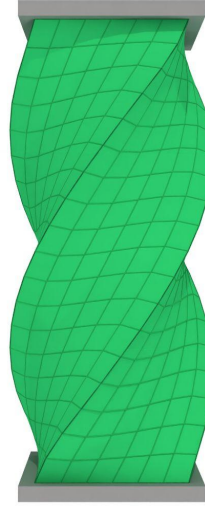
P2



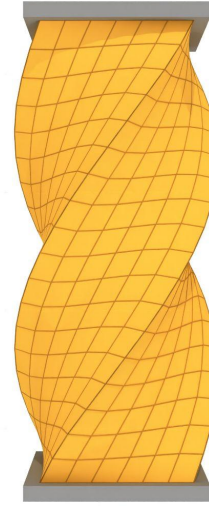
P3



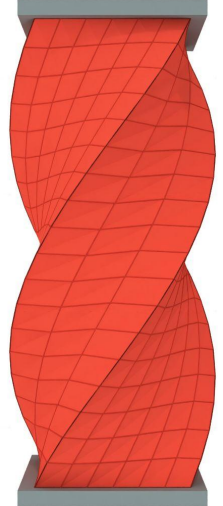
Q1



Prisme



Pyramide



Conclusion

	XPBD	VBD
Approche locale	Eléments	Sommets
Contrainte forte	Oui	Non (mais)
Numériquement robuste	Stable	Stable (mais)
Précision numérique	Limitée	Bonne
Contionnement	Masses	Raideurs
Ordre des dérivées	1	2
Couplée	Non (mais)	Oui

Lois de comportements et matériaux

Néo-Hooke stable [Smith 2018]

$$\Psi(F) = \underbrace{\frac{\mu}{2}(tr(F^T F) - 3)}_{\text{Étirement}} + \underbrace{\frac{\lambda}{2}(det(F) - \alpha)^2}_{\text{Volume}}$$

Évalue la densité d'énergie élastique

Coefficients de Lamé

$$\mu = \frac{E}{2(1 + \nu)} \quad \lambda = \frac{E\nu}{(1 + \nu)(1 - 2\nu)}$$

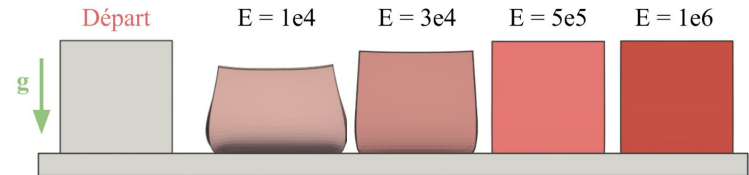
Module de Young
 $E \in [0, +\infty[$

Coefficient de poisson
 $\nu \in [0, 0.5[$

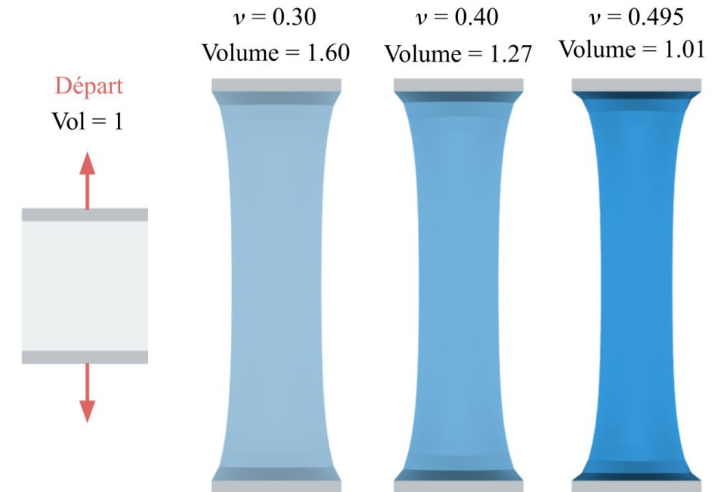
Si conservation de volume parfaite alors $\lambda \rightarrow +\infty$

Plus les paramètres de raideurs sont élevés,
plus le problème est difficile

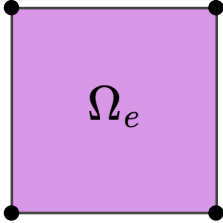
Module de Young

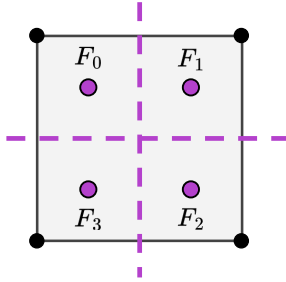


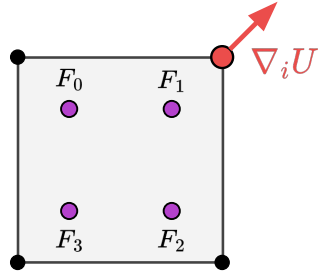
Coefficient de Poisson



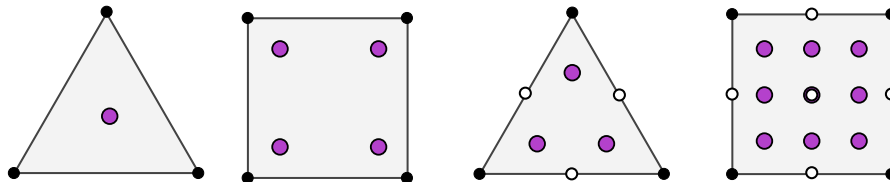
Évaluation des grandeurs physiques

Énergie élastique
$U(F) = \int_{\Omega_e} \Psi(F) dX$
 A square element domain Ω_e is shown, shaded in light purple, with black dots at its four corners.
Calcul de l'énergie élastique sur le domaine de l'élément

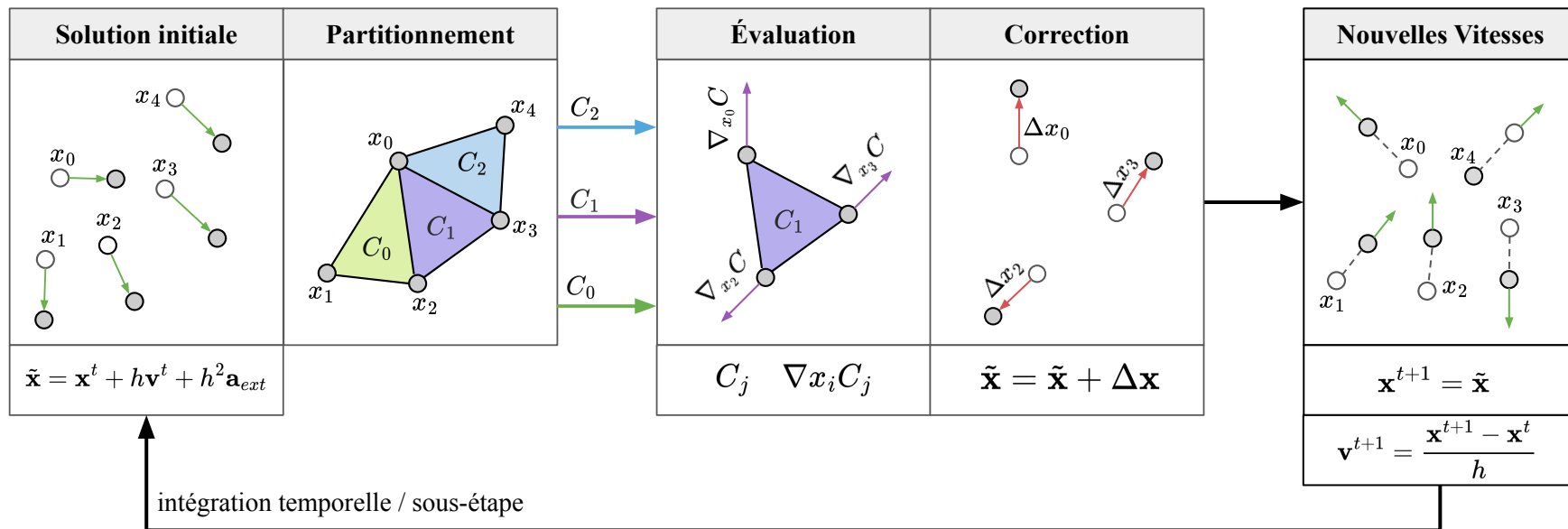
Quadrature
$U(F) = \sum_q^Q V_q \Psi(F_q)$
 A square element is shown with four quadrature points F_0, F_1, F_2, F_3 marked by purple dots. Dashed purple lines divide the square into four quadrants.
Évaluation de la déformation à des positions clés

Gradient énergie
$\nabla_i U = \frac{\partial U}{\partial x_i} = \sum_q^Q V_q \frac{\partial \Psi(F_q)}{\partial x_i}$
 A square element is shown with four quadrature points F_0, F_1, F_2, F_3 marked by purple dots. A red arrow labeled $\nabla_i U$ points from the top-right corner, representing the energy gradient.
Permet de définir les forces en chaque sommet

#Quadratures
Type & Ordre

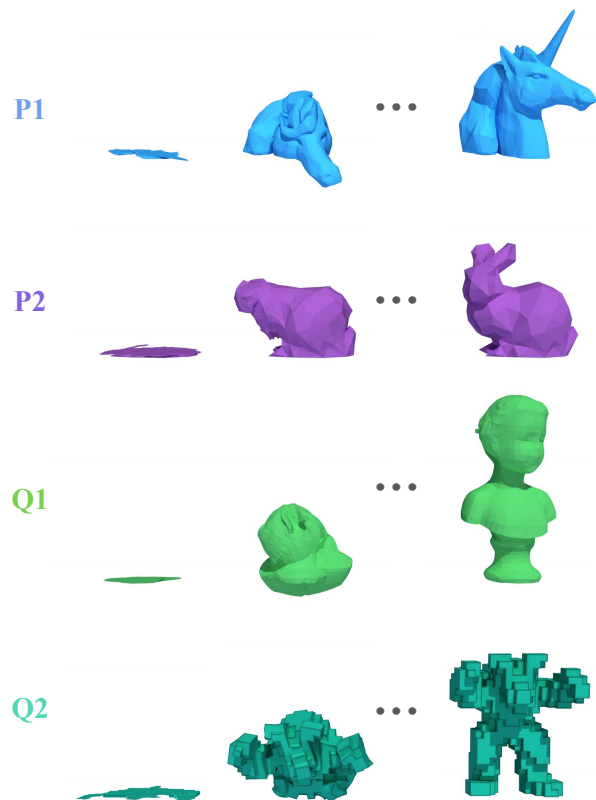


Résolution locale avec XPBD

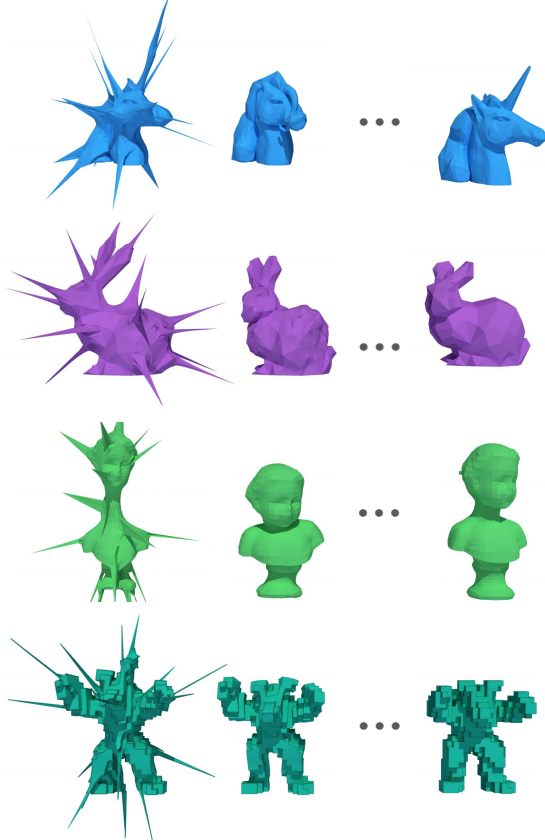


VBD : Robustesse en grandes déformations

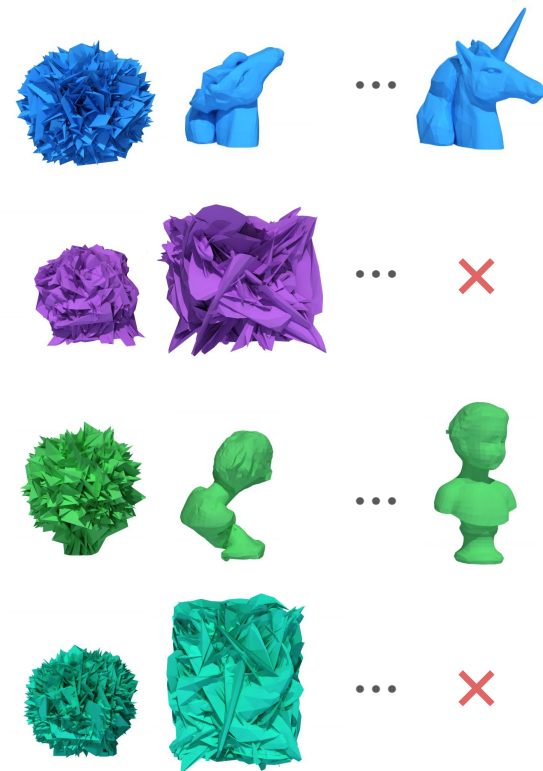
Aplatissement sur plan



Étirement local

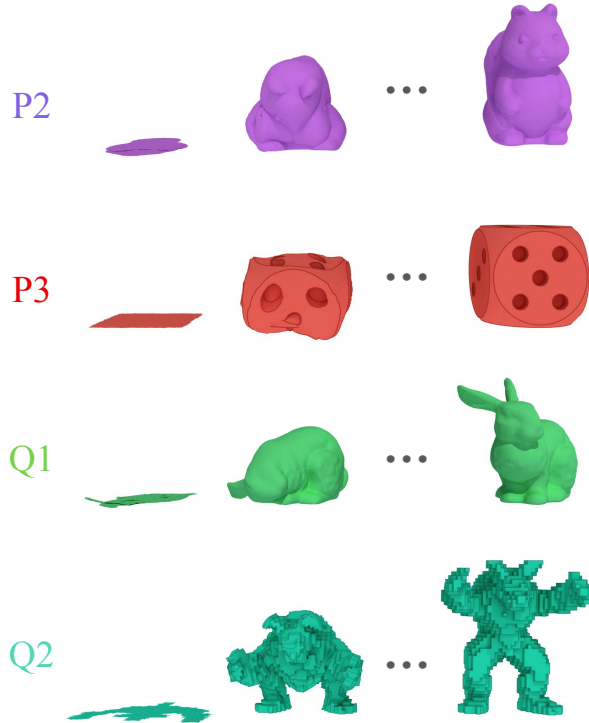


Aléatoire

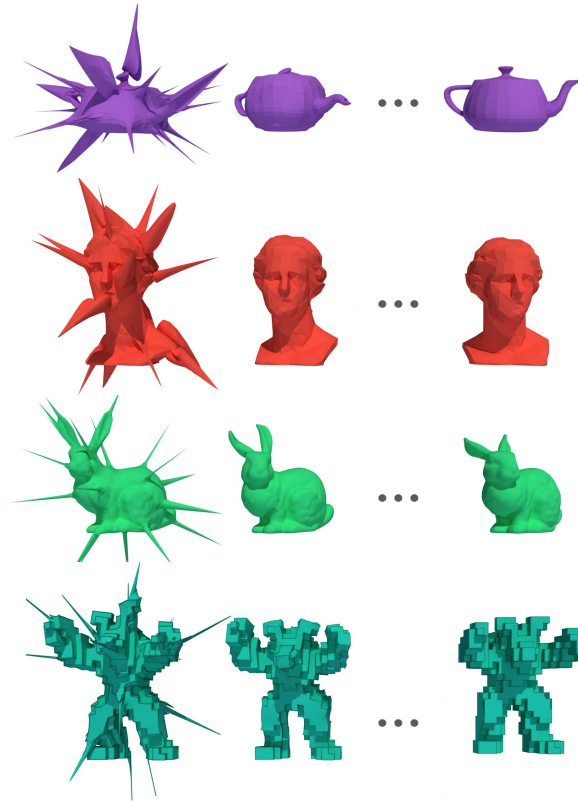


XPBD : Robustesse en grandes déformations

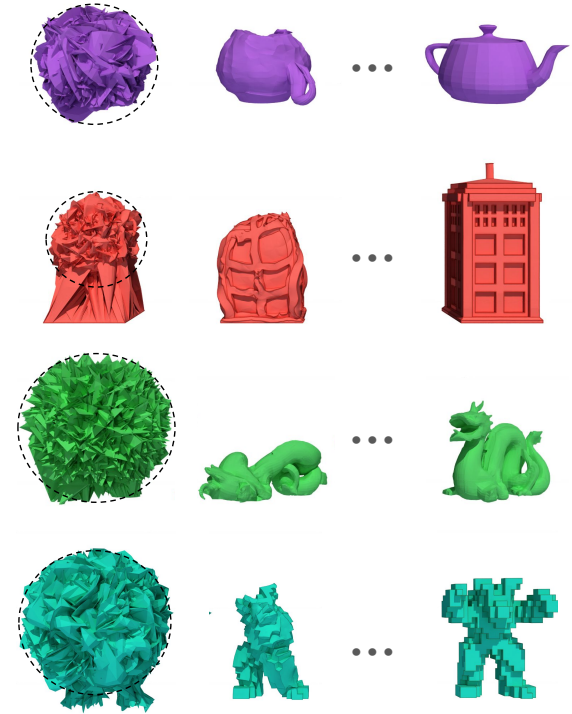
Aplatissement sur plan



Étirement local

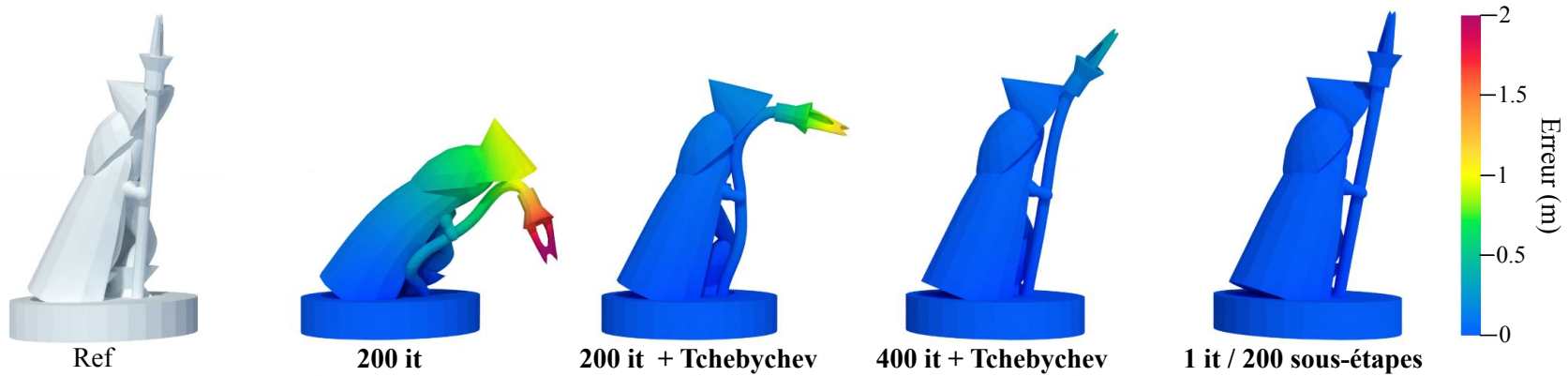


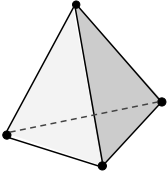
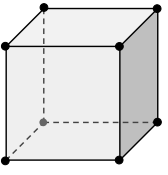
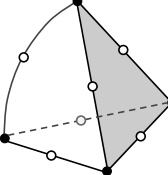
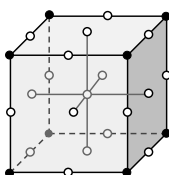
Aléatoire



Les déformations complexes peuvent être résolues même en ordre élevé avec *XPBD*

VBD : Sous-étapes pour accélérer les méthodes locales



	Coût à convergence [test de flexion]			
				
Tchebychev	138 ms	96.7 ms	373 ms	550 ms
Sous-étapes	12.3 ms	17.3 ms	28 ms	103 ms

Résolution locale plus efficaces

5-10 fois plus rapide

Pas de “bruit”

Possible d'utiliser des itérations

